



Form2
Capstone Design System Analysis
Design and Development of a WhatsApp-
Based Issue Reporting and Analytics
Dashboard System for TRAMOS Web at PT
Andakara Informatika Nusantara

GROUP MEMBER:

No.	Student Name	Student ID	Study Program
1.	Cayla Winri Azzahra	012202300098	Information System
2.	Ivander Johana Pratama	012202300030	Information System

Advisor : Hadi Suprayitno, S.Kom., M.M.

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

TABLE OF CONTENT

A. EXISTING AND PROPOSED SYSTEM	5
1. Existing Business Process.....	5
2. Proposed Business Process	6
B. GLOBAL DESCRIPTION OF THE PRODUCT.....	9
1. Main Functionality.....	9
2. User Characteristics	9
3. Constraints	10
4. Product Development Environment.....	10
C. REQUIREMENT ANALYSIS	12
1. Interface	12
2. Functional Description.....	12
3. Data Requirement from user's perspective.....	18
4. Functional Requirement.....	32
5. Non-Functional Requirement.....	37
D. ALTERNATIVE SOLUTION.....	38
REFERENCES	41

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

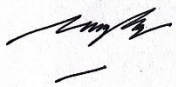
Name : 1. Cayla Winri Azzahra – 012202300098
2. Ivander Johana Pratama - 012202300030

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled “**Design and Development of a WhatsApp-Based Issue Reporting and Analytics Dashboard System for TRAMOS Web at PT Andakara Informatika Nusantara**” is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, 28 February 2026

Signer 1	Signer 2
	
Cayla Winri Azzahra - 012202300098	Ivander Johana Pratama - 012202300030

Design and Development of a WhatsApp-Based Issue Reporting and Analytics Dashboard System for TRAMOS Web at PT Andakara Informatika Nusantara

Approved:

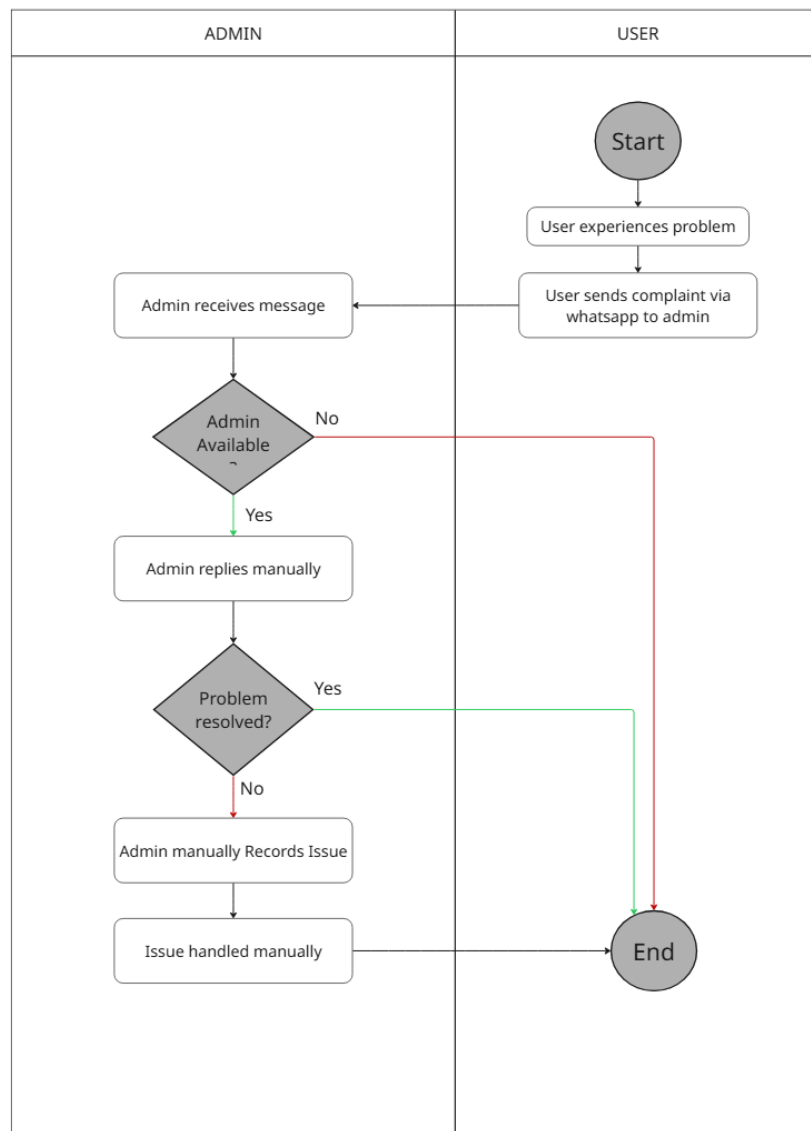
Hadi Suprayitno,
S.Kom., M.M.
Capstone Advisor

Ronny Juwono, S.Pd., M.T
Program Head of Information System

Prof. Dr. Ir. Wiranto Herry Utomo, M.Kom.
Dean of Faculty of Computer Science

A. EXISTING AND PROPOSED SYSTEM

1. Existing Business Process



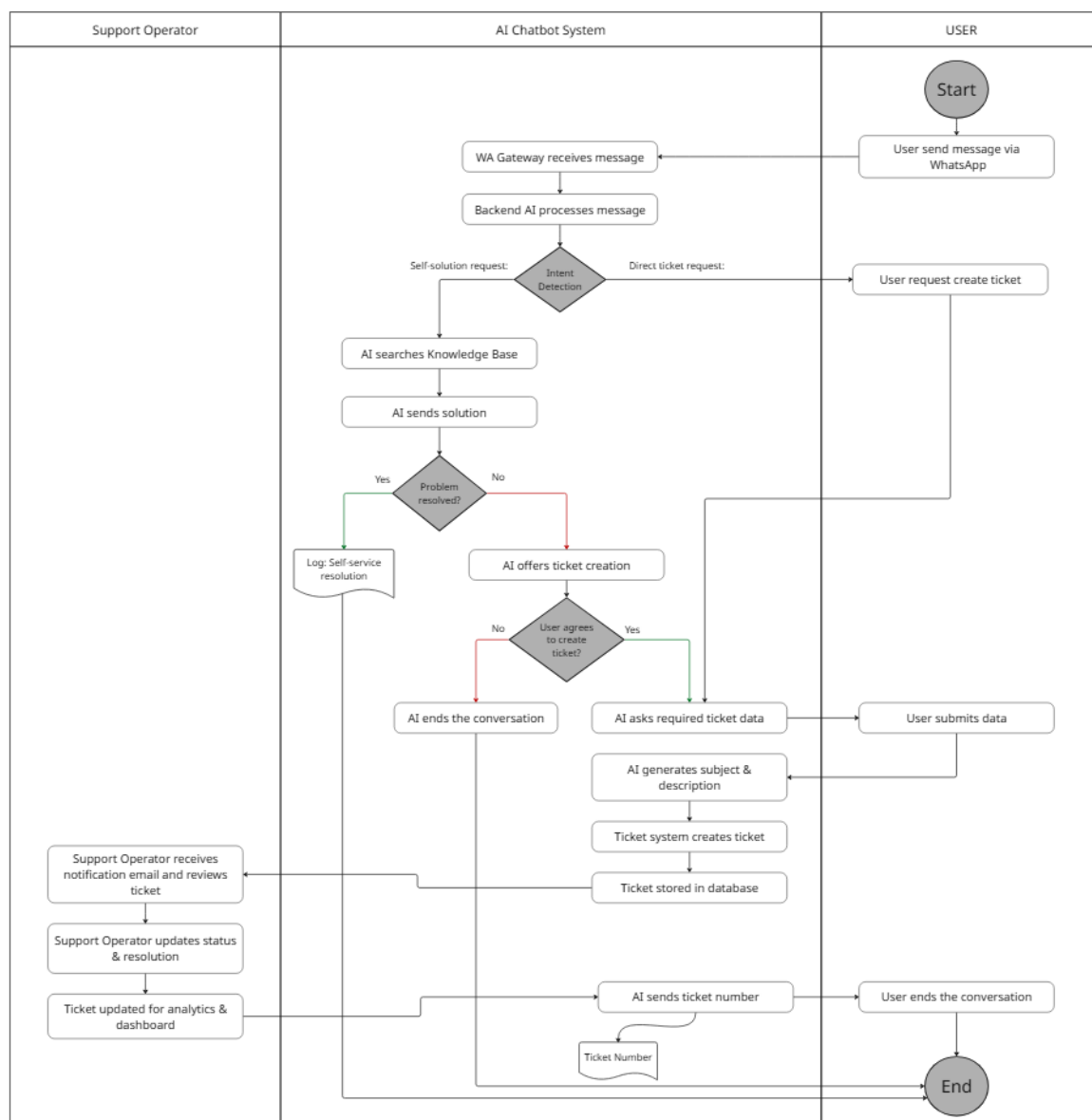
In this current condition at PT Andakara Informatika Nusantara, the issue reporting process in TRAMOS Web system is still done manually and relies on a single support person. The process begins when a user (driver / staff) having an obstacle or problem while performing their duties. The user then sends a complaint by sending a message directly to the person via WhatsApp.

Issues are handled one by one based on availability of the person/admin. If the admin is unavailable, messages from users cannot be handled immediately and users must wait until admin can respond. This situation has the potential to cause delays in handling issues, especially when the number of reports received is high. Then, if admin is available, they will respond manually via WhatsApp.

Next, the reported issue will be evaluated to determine whether it has been resolved. If the issue can be resolved through communication, the reporting process will end without any structured record in the system. However, if the issue has not been resolved, admin will manually record the issue and take further action based on the available information. The entire process of recording and handling problems still depends on the individual efforts of the admin, so there is no centralized recording system, ticket tracking, or statistical reports that can be used for evaluation or decision-making by management.

2. Proposed Business Process

A. AI Chatbot & Ticketing System



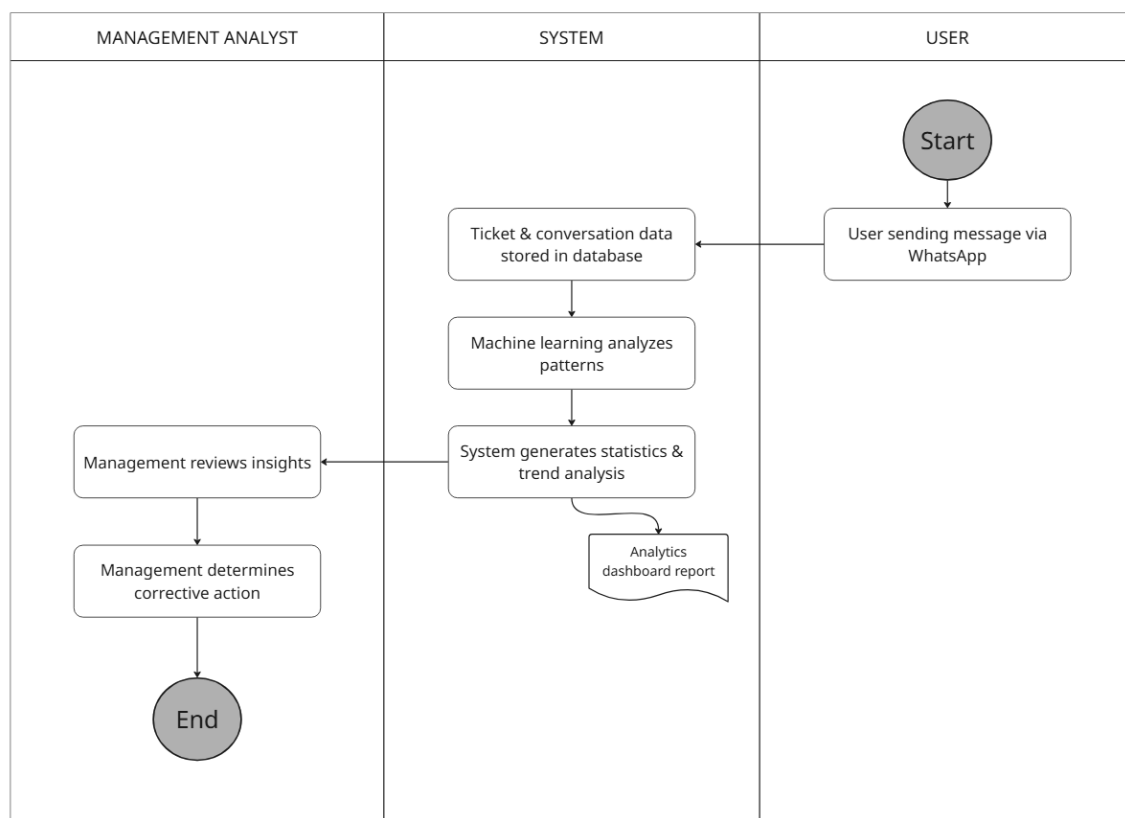
The business process proposed in the TRAMOS Web System integrates a WhatsApp-based AI Chatbot to handle reporting and issue resolution automatically. In this process, users

send problem reports via WhatsApp, which are then processed by the AI system to identify the intent of the message sent.

Based on the analysis result, system can provide solutions directly through the knowledge base or proceed to the ticket creation process if necessary. If the problem cannot be resolved through an automated solution, the system will offer to create a ticket and collect supporting data from the user. The data is then processed to automatically generate a ticket that is stored in the ticketing system. The ticket number will sent back to the user as proof that the report has been recorded and will be followed up.

Immediately after ticket creation, the system automatically sends an email notification to all assigned support operators. This email contains ticket information and also includes a direct link to the osTicket dashboard. The Support Operator is responsible for reviewing tickets, updating statuses, and recording the results of problem resolutions. All ticket updates will be synchronized with the system for analytics and reporting purposes. With this process, problem reporting no longer relies on manual interaction with admin, but can be handled in real-time, in a structured manner, and well-documented through the system.

B. Dashboard & Machine Learning Analytics



In this business process, all ticket data and conversations from user interactions via WhatsApp are stored centrally in the system database. The collected data is then processed by

a machine learning module to analyze problem patterns, frequency of occurrence, and trends that emerge over time.

Based on the analysis result, the system generates statistical reports and visualizations in the form of an analytics dashboard. This dashboard shows information relevant to management, such as the most common types of problems and problem trends within a given period. Then, management can review the insights available on the dashboard and determine the corrective actions or improvement strategies needed to enhance the system's performance and operational reliability.

B. GLOBAL DESCRIPTION OF THE PRODUCT

1. Main Functionality

The product developed is a WhatsApp-based problem reporting and analysis system integrated with TRAMOS Web. This system is designed to support the process of reporting, recording, and analyzing operational problems automatically and in a structured manner. The main functions of this system include:

a. Automated Issue Reporting

The system allows users to report operational issues directly via WhatsApp without having to access the website. Messages sent will be processed by an AI Chatbot to understand the context and type of issue reported.

b. AI-Based Problem Analysis and Response

The AI system analyzes user messages to detect intent and classify issues. Based on the analysis result, the system can provide automated solutions sourced from the TRAMOS knowledge base.

c. Automated Ticket Creation

If the problem cannot be resolved through an automated solution, the system will facilitate the automatic creation of ticket by collecting important data from the user and storing it in the ticketing system.

d. Analytics and Insight Dashboard

The system provides an analytics dashboard that displays statistics, trends and problem distribution to support monitoring and evaluation by management.

e. Machine Learning-Based Pattern Analysis

Historical data collected is analyzed using a machine learning approach to identify recurring problem patterns and support data driven decision making.

f. Automated Email Notifications to Support Operators

This feature ensures operators receive immediate alerts for new ticket, reducing response time and improving ticket handling efficiency without requiring manual dashboard monitoring.

2. User Characteristics

Users	Responsibility	Access rights	Education Levels	Skill Levels	Type of Training
-------	----------------	---------------	------------------	--------------	------------------

User (Driver / Staff)	Report operational issues and provide supporting information through WhatsApp	Send messages via WhatsApp, receive automated responses, receive ticket number	High School - Diploma	Basic smartphone usage	Short chat-based demo (±30 minutes)
Support Operator	Review tickets, update ticket status, add resolution notes through ticketing system	Access to osTicket dashboard for ticket management	Diploma - Bachelor	Intermediate	System usage training (±30 minutes)
Management Analyst	Monitor analytics dashboard, analyze trends, and determine corrective actions	Access to analytics dashboard and reports	Diploma – Bachelor	Advanced analytical skills	Dashboard & analytics overview (±30 minutes)

3. Constraints

From the user's perspective, this system has several limitations that need to be considered. The system is highly dependent on WhatsApp as its primary means of communication, so users must have an active WhatsApp account and a stable internet connection. Network limitations can cause delays in system response under certain conditions.

In addition, the knowledge base was initially developed in Indonesian, so system does not yet fully support multilingual use. The automated solutions provided by AI are also limited to simple and repetitive problems that are covered in the knowledge base. For complex technical problems, human action is still required.

This system is also designed with mobile-first approach, so most of its functions are focused on smartphone users in the field. Non-mobile devices is not a top priority in the initial stages of development.

4. Product Development Environment

a. Hardware

- **Laptop**

Laptop are used as the main device in the system development process, such as to write program code, run local servers, manage Docker containers, and perform backend and dashboard analytics testing. The minimum specifications required include an Intel Core i5 or AMD Ryzen 5 equivalent processor, a minimum of 8 GB of RAM, and SSD-based storage to ensure fast and stable performance.

- **Smartphone**
Smartphones are used to test system integration with WhatsApp directly. Testing on actual devices is necessary to ensure that the chatbot communication flow works properly, messages are received and sent in real time, and user interactions can take place smoothly. The use of smartphones also helps identify obstacles that may not be detected when testing using desktop devices.

b. Software

- **Visual Studio Code**
VS Code is used as the primary Integrated Development Environment (IDE) in system development. VS Code supports various programming languages and frameworks used in this project, and provides important features such as syntax highlighting, debugging tools, and Git integration for code version management.
- **Figma**
Figma is used as a UI/UX design tool to design the appearance of analytical dashboards and system interaction flows. This platform supports the creation of wireframes and interactive prototypes that facilitate system visualization prior to the implementation stage.
- **Docker**
Docker is used to create isolated and consistent development environments. By using containers, all system components such as backend services and databases can be run in a structured manner without depending on the local operating system configuration.
- **PostgreSQL**
PostgreSQL is used as a database management system to store conversation data, tickets, and analytical data. We choose this database was chosen because it performs well, supports large data scales, and can be integrated with various backend frameworks.
- **Gemini AI API**
Gemini AI API is used to support natural language processing in AI chatbots. This API helps the system understand user messages, classify issues, and automatically summarize tickets based on user input.
- **WhatsApp Gateway API**
WhatsApp Gateway API is used as a connector between the backend system and the WhatsApp application. This API allows the system to receive messages from users and send chatbot responses automatically.
- **SMTP Email Service (Gmail)**
Gmail SMTP is used to provide automated email notification functionality.

c. Connection

A stable and fast internet connection is essential during the system development process. This connection is used to access external APIs, run WhatsApp webhooks, manage code repositories, and perform real-time integration testing. An unstable internet connection can affect the system development and testing process, making a reliable connection a key requirement.

C. REQUIREMENT ANALYSIS

1. Interface

The interface on this WhatsApp-Based Issue Reporting and Analytics Dashboard system is designed to support interactions between users, the backend system, and management in an efficient and easy-to-use manner.

a. Hardware

- **Smartphone**
Used by users (drivers/staff) to send problem reports and interact with the AI Chatbot via the WhatsApp application.
- **Laptop / Desktop Computer**
Used by management and operators to access the analytics dashboard and monitor report data via a web browser.

b. Software

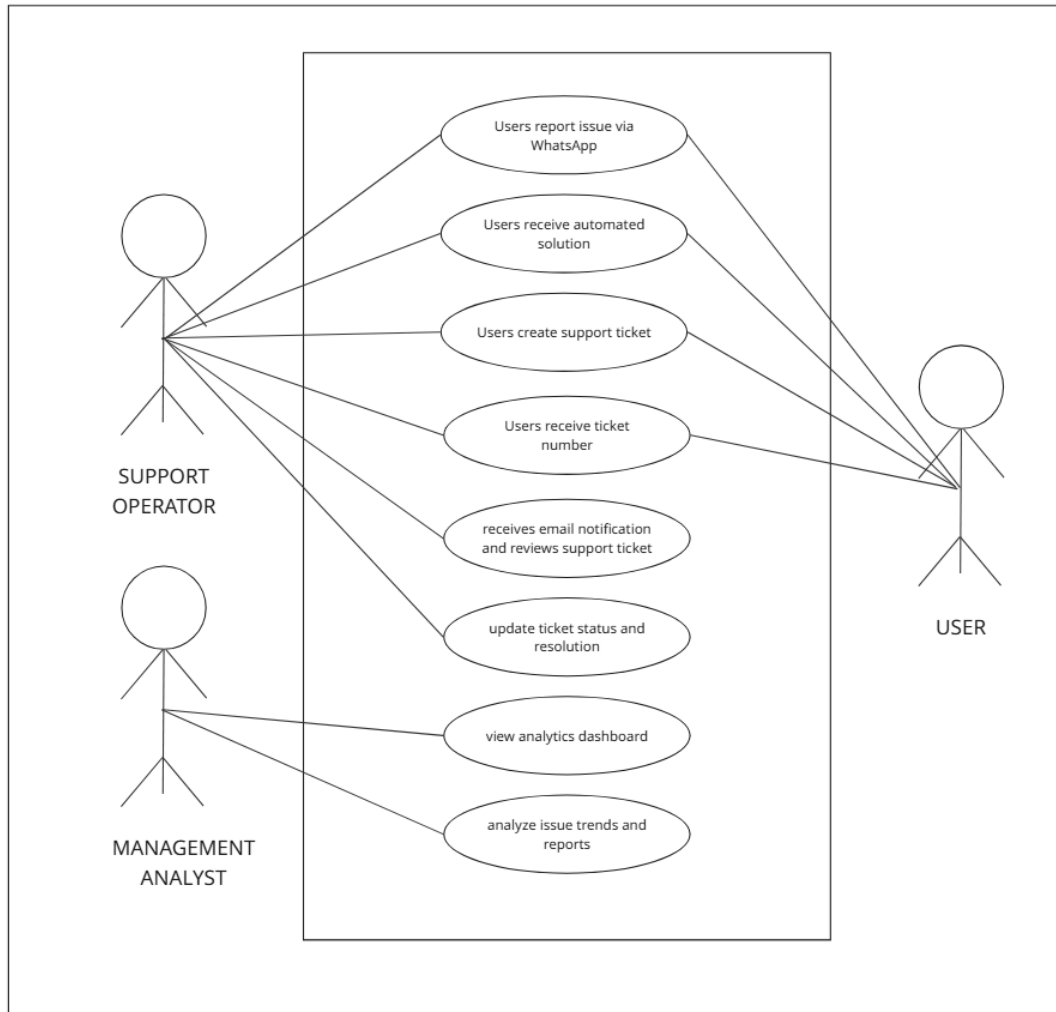
- **WhatsApp Mobile Application**
Serves as the main interface for users to report issues without having to access additional applications or websites.
- **Web-Based Analytics Dashboard**
Used by management and operators to view statistical reports, problem trends, and insights from data analysis.
- **Backend REST API (FastAPI)**
Acts as a connector between the WhatsApp Gateway, ticketing system, database, and analytical dashboard.
- **osTicket System**
Used for structured ticket management and integrated with the AI Chatbot system.

c. Connection

- **HTTPS Webhook Connection**
Used to receive and send messages between WhatsApp Gateway and the backend system.
- **Secure Internet Connection**
Required to ensure real-time and secure data communication.

2. Functional Description

a. Use case diagram



b. Use case scenario

i. Use case: Users report issue via WhatsApp

Main Actor	User (Driver / Staff)
Description	Describes the process when a user reports an operational issue via WhatsApp to the AI Chatbot System
Initial Condition	• User has an active WhatsApp account • An internet connection is available • AI Chatbot System is active
Main Success Scenario	1. User send a message containing the problem report via WhatsApp

	2. The system receives the message via WhatsApp Gateway 3. The system processes the message and stores it in the database 4. The system proceeds to the internet analysis and problem handling process
Alternative Scenario	If the internet connection is instable, the message is delayed until the connection returns to normal.
Final Condition	The problem report is successfully received and recorded in the system.

ii. Use case: Users receive automated solution

Main Actor	User (Driver / Staff)
Description	Describes the process when the system provides an automated solution based on the TRAMOS knowledge base.
Initial Condition	• User has submitted a problem report • System successfully detects the intent of the message
Main Success Scenario	1. System identifies the problems as an issue available in the knowledge base 2. The system sends an automated solution to the user via WhatsApp 3. The system asks whether the problem has been resolved
Alternative Scenario	If the solution is not relevant, the system offers to create a ticket.
Final Condition	The problem is resolved through self-service or proceeds to ticket creation.

iii. Use case: Users create support ticket

Main Actor	User (Driver / Staff)
Description	Describes the process of creating a support ticket when an issue cannot be resolved automatically.
Initial Condition	• The user's issue has not been resolved • The system offers the option to create a ticket
Main Success Scenario	1. The user agrees to create a ticket 2. The system request supporting data (e.g., vehicle license plate, location, time of incident) 3. The system automatically generates a ticket and saves it to the ticketing system
Alternative Scenario	The user declines to create a ticket, and the conversation is ended by the system
Final Condition	The ticket is successfully created and saved to the database

iv. Use case: Users receive ticket number

Main Actor	User (Driver / Staff)
Description	Describes the process of delivering the ticket number to the user after the ticket has been successfully created
Initial Condition	• Ticket has been successfully created by the system
Main Success Scenario	1. System sends the ticket number to the user via WhatsApp 2. User receives the ticket number as a reference for reporting
Alternative Scenario	If the messages delivery is delayed, the system will retry the delivery
Final Condition	User receives the ticket number and the reporting process is complete

v. Use case: Receives email notification and reviews support ticket

Main Actor	Support Operator
Description	Describes the process when a Support Operator receives an automated email notification about a new ticket and reviews it through the osTicket dashboard
Initial Condition	• Tickets are available in the ticketing system • Operational email is configured in the system • Support Operator's email is active
Main Success Scenario	1. System automatically sends email notification to operator 2. Operator receives email notification within 10-30 seconds 3. Operator opens the email and reviews ticket details 4. Operator clicks the link in email to open osTicket dashboard 5. Operator opens the ticket and reviews complete details
Alternative Scenario	Operator directly open osTicket without using email link
Final Condition	The ticket is ready to be processed or followed up on

vi. Use case: Update ticket status and resolution

Main Actor	Support Operator
Description	Describes the process of updating ticket status and resolution
Initial Condition	• Support Operator is handling a ticket
Main Success Scenario	1. The Support Operator updates the ticket status (Open / In Progress / Resolved) 2. The Support Operator adds a resolution note 3. The system saves the ticket update to the database

Alternative Scenario	The ticket is transferred to another party for further handling
Final Condition	The ticket status is updated and ready for analysis

vii. Use case: View analytics dashboard

Main Actor	Management Analyst
Description	Describes the process of monitoring the analytics dashboard by management
Initial Condition	• Ticket and conversation data is available in the database • The Management Analyst has access rights to the dashboard
Main Success Scenario	1. The Management Analyst accesses the analytics dashboard 2. The system displays statistics and data visualizations of issues
Alternative Scenario	The data is incomplete, so the system displays partial data
Final Condition	The dashboard is successfully displayed to the Management Analyst

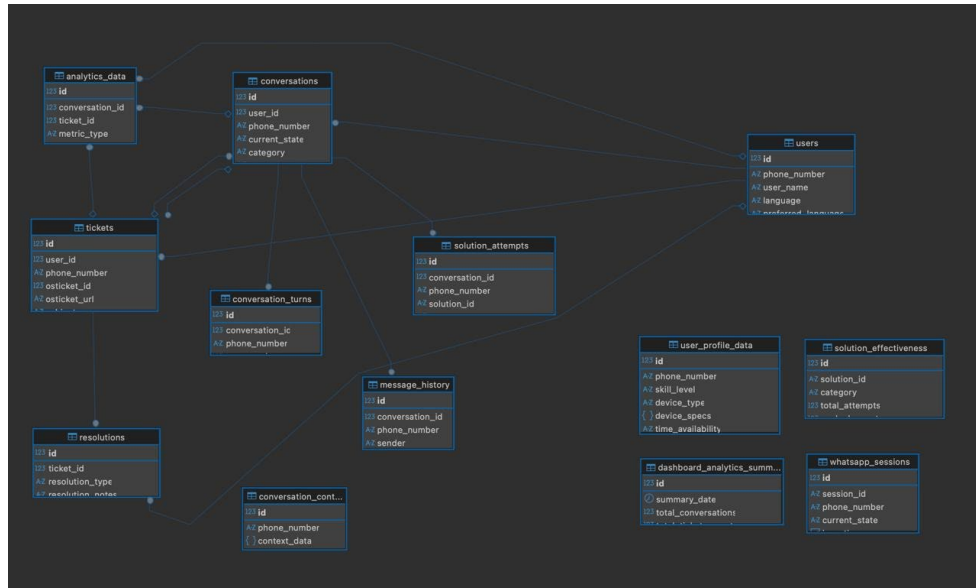
viii. Use case: Analyze issue trends and reports

Main Actor	Management Analyst
Description	Describes the process of analyzing trends and making data driven decisions
Initial Condition	• The analytics dashboard is displayed
Main Success Scenario	1. The Management Analyst analyzes issue trends 2. The Management Analyst identifies patterns and potential improvements 3. The Management Analyst determines corrective actions

Alternative Scenario	The data does not show any significant patterns, so no immediate action is taken
Final Condition	The insights are used as a basis for management decision-making

3. Data Requirement from user's perspective

a. ERD (Entity Relationship Diagram)



b. Data Dictionary

i. Users

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each user	1
2	phone_number	VARCHAR	WhatsApp phone number (e.g., 6281234567890)	6281234567890
3	user_name	VARCHAR	Display name of the user (driver name)	Budi Santoso
4	role	VARCHAR	User role: 'user', 'operator', 'analyst', 'admin'	user
5	department	VARCHAR	Department or team assignment within the organization	Fleet Operations
6	language	VARCHAR	Detected language of the user	id
7	preferred_language	VARCHAR	User's preferred language for communication	id

8	is_active	BOOLEAN	Whether the user account is currently active	TRUE
9	first_contact_at	TIMESTAMP	Timestamp of the user's first interaction with the system	2026-01-15 8:30:00
10	last_contact_at	TIMESTAMP	Timestamp of the user's most recent interaction	2026-02-20 14:22:00
11	total_messages	INTEGER	Total number of messages sent by this user	47
12	total_tickets	INTEGER	Total number of support tickets created by this user	5
13	resolved_tickets	INTEGER	Number of tickets that have been resolved for this user	4
14	created_at	TIMESTAMP	Record creation timestamp	2026-01-15 8:30:00
15	updated_at	TIMESTAMP	Record last update timestamp	2026-02-20 14:22:00

ii. Conversations

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each conversation	101
2	user_id	INTEGER	Reference to the user who owns this conversation	1
3	phone_number	VARCHAR	WhatsApp phone number of the user	6281234567890
4	current_state	VARCHAR	Current state of the conversation (e.g., new_issue, analyzing, resolved, closed)	collecting_details
5	category	VARCHAR	Issue category (e.g., GPS, Camera, Battery, Network)	GPS
6	intent	VARCHAR	Detected user intent (e.g., troubleshooting, escalate, resolved)	troubleshooting
7	issue_description	TEXT	Full description of the reported issue	GPS tidak bisa connect di unit B 7821
8	issue_start_time	TIMESTAMP	When the issue was first reported by the user	2026-02-20 9:15:00
9	ticket_id	INTEGER	Reference to the associated support ticket (set after ticket creation)	201

10	context_data	JSON	Flexible JSON storage for conversation context data	{"step": 3, "retries": 0}
11	meta_data	JSON	Additional metadata in JSON format	{"source": "whatsapp"}
12	created_at	TIMESTAMP	Conversation creation timestamp	2026-02-20 9:15:00
13	updated_at	TIMESTAMP	Last update timestamp	2026-02-20 9:32:00
14	last_message_at	TIMESTAMP	Timestamp of the last message in this conversation	2026-02-20 9:32:00

iii. Message_history

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each message	1001
2	conversation_id	INTEGER	Reference to the parent conversation	101
3	phone_number	VARCHAR	WhatsApp phone number of the sender/receiver	6281234567890
4	sender	VARCHAR	Who sent the message: 'user', 'bot', or 'system'	user
5	message_text	TEXT	The actual text content of the message	GPS saya tidak bisa connect
6	message_type	VARCHAR	Type of message content: 'text', 'image', 'document', 'location', 'audio', 'video'	text
7	language	VARCHAR	Detected language of the message	id
8	intent	VARCHAR	AI-detected intent of the message	troubleshooting
9	category	VARCHAR	AI-detected issue category	GPS
10	confidence	INTEGER	AI confidence score (0–100)	85
11	message_id	VARCHAR	WhatsApp message ID for deduplication	wamid.HBgNNjI4MTIzNDU2Nzg5
12	meta_data	JSON	Additional metadata in JSON format	{"processed": true}

13	created_at	TIMESTAMP	Message timestamp	2026-02-20 9:16:30
----	------------	-----------	-------------------	--------------------

iv. Tickets

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each ticket	201
2	user_id	INTEGER	Reference to the user who created the ticket	1
3	phone_number	VARCHAR	WhatsApp phone number of the ticket creator	6281234567890
4	osticket_id	INTEGER	External osTicket system ticket ID	45823
5	osticket_url	VARCHAR	URL to the ticket in the osTicket web interface	https://support.andakara.com/scp/tickets.php?id=45823
6	subject	VARCHAR	Ticket subject/title	[GPS] GPS tidak bisa connect - B 7821
7	description	TEXT	Detailed description of the issue	Driver Budi melaporkan GPS tidak connect sejak pagi di unit B 7821, lokasi Tol Cikampek KM 45
8	category	VARCHAR	Issue category (e.g., GPS, Camera, Battery)	GPS
9	status	VARCHAR	Ticket status: 'open', 'in progress', 'on hold', 'resolved', 'closed', 'pending'	open
10	priority	VARCHAR	Priority level: 'low', 'medium', 'high', 'critical'	high
11	conversation_id	INTEGER	Reference to the originating conversation	101
12	resolution_notes	TEXT	Notes about how the ticket was resolved	GPS module di-restart oleh teknisi lapangan
13	resolved_at	TIMESTAMP	Timestamp when the ticket was resolved	2026-02-20 11:45:00
14	resolution_time_minutes	INTEGER	Total time from creation to resolution in minutes	150
15	user_satisfaction	INTEGER	User satisfaction rating (1–5 scale)	4
16	feedback_text	TEXT	User's textual feedback about the resolution	Terima kasih, GPS sudah normal kembali
17	created_at	TIMESTAMP	Ticket creation timestamp	2026-02-20 9:30:00
18	updated_at	TIMESTAMP	Last update timestamp	2026-02-20 11:45:00

v. Resolutions

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each resolution	301
2	ticket_id	INTEGER	Reference to the resolved ticket	201

			(one resolution per ticket)	
3	resolution_type	VARCHAR	How the issue was resolved: 'ai_solution', 'operator_resolved', 'user_resolved', 'escalated'	ai_solution
4	resolution_notes	TEXT	Detailed notes about the resolution process	AI menyarankan restart GPS module, user konfirmasi berhasil
5	resolved_by_user_id	INTEGER	Reference to the operator who resolved the ticket	NULL
6	resolved_at	TIMESTAMP	Timestamp of the resolution	2026-02-20 9:28:00
7	resolution_time_minutes	INTEGER	Total resolution time in minutes	13
8	ai_attempted	BOOLEAN	Whether the AI chatbot attempted to resolve the issue	TRUE
9	ai_successful	BOOLEAN	Whether the AI chatbot successfully resolved the issue	TRUE
10	ai_confidence	DOUBLE PRECISION	AI's confidence score in the solution provided	0.87
11	meta_data	JSONB	Additional metadata in JSON format	{"solution_id": "gps_restart_01"}
12	created_at	TIMESTAMP	Record creation timestamp	2026-02-20 9:28:00
13	updated_at	TIMESTAMP	Record last update timestamp	2026-02-20 9:28:00

vi. Analytics_data

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each analytics record	501
2	conversation_id	INTEGER	Reference to the related conversation (if applicable)	101
3	ticket_id	INTEGER	Reference to the related ticket (if applicable)	201
4	metric_type	VARCHAR	Type of metric: 'resolution_time', 'ai_success_rate', 'ticket_volume', 'category_count', 'operator_performance', 'user_satisfaction'	resolution_time
5	metric_value	DOUBLE PRECISION	Numeric value of the metric	13.0

6	category	VARCHAR	Issue category this metric belongs to	GPS
7	date_recorded	DATE	Date when the metric was recorded	2026-02-20
8	hour_recorded	INTEGER	Hour of the day (0–23) for hourly tracking	9
9	operator_id	INTEGER	Reference to the operator for performance tracking	5
10	meta_data	JSONB	Additional context data in JSON format	{"unit": "minutes"}
11	created_at	TIMESTAMP	Record creation timestamp	2026-02-20 9:28:00

vii. Conversation_context

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each context record	601
2	phone_number	VARCHAR	WhatsApp phone number (one context per phone number)	6281234567890
3	context_data	JSON	Full conversation context stored as JSON	{"driver_name": "Budi", "unit": "B 7821"}
4	current_state	VARCHAR	Cached current conversation state	collecting_details
5	category	VARCHAR	Cached issue category	GPS
6	issue_description	TEXT	Cached issue description	GPS tidak bisa connect
7	last_intent	VARCHAR	Most recently detected user intent	troubleshooting
8	created_at	TIMESTAMP	Record creation timestamp	2026-02-20 9:15:00
9	updated_at	TIMESTAMP	Record last update timestamp	2026-02-20 9:20:00
10	last_accessed_at	TIMESTAMP	Last time this context was accessed	2026-02-20 9:22:00

viii. Conversation_turns

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each turn	701
2	conversation_id	INTEGER	Reference to the parent conversation	101
3	phone_number	VARCHAR	WhatsApp phone number	6281234567890
4	turn_number	INTEGER	Sequential number of the turn in the conversation (1st, 2nd, etc.)	3

5	user_message	TEXT	The user's message in this turn	GPS saya tidak bisa connect
6	user_intent	VARCHAR	AI-detected intent from the user's message	troubleshooting
7	user_category	VARCHAR	AI-detected issue category from the user's message	GPS
8	user_confidence	INTEGER	AI confidence in intent/category detection (0–100)	85
9	bot_response	TEXT	The chatbot's response in this turn	Baik, coba restart GPS module di unit Anda...
10	bot_state	VARCHAR	Conversation state after this turn was processed	offering_solutions
11	processing_time_ms	INTEGER	Time taken to process this turn in milliseconds	1250
12	turn_type	VARCHAR	Classification of the turn: 'greeting', 'troubleshooting_step', 'escalation', etc.	troubleshooting_step
13	created_at	TIMESTAMP	Turn timestamp	2026-02-20 9:18:00

ix. Whatsapp_sessions

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each session record	801
2	session_id	VARCHAR	Unique session identifier string	session_6281234567890_1708412100
3	phone_number	VARCHAR	WhatsApp phone number	6281234567890
4	current_state	VARCHAR	Current dialog flow state (e.g., GREETING, COLLECTING_NAME, COLLECTING_PROBLEM, CONFIRMING_DETAILS, CLOSED)	COLLECTING_PROBLEM
5	is_active	BOOLEAN	Whether the session is currently active	TRUE
6	driver_name	VARCHAR	Name of the driver/user collected during the dialog	Budi Santoso
7	problem_description	TEXT	Description of the problem reported by the user	GPS tidak bisa connect sejak pagi
8	problem_category	VARCHAR	AI-detected category of the problem	GPS
9	problem_severity	VARCHAR	Severity level of the problem: 'critical', 'high', 'medium', 'low'	high
10	vehicle_unit	VARCHAR	Vehicle unit/fleet number reported by the user	B 7821
11	location	VARCHAR	Location where the issue occurred	Tol Cikampek KM 45

12	issue_time	VARCHAR	Time when the issue occurred (as reported by user)	8:00
13	ticket_created	BOOLEAN	Whether a support ticket has been created for this session	FALSE
14	ticket_id	VARCHAR	Internal ticket ID reference	201
15	osticket_id	INTEGER	External osTicket system ticket ID	45823
16	message_count	INTEGER	Total number of messages exchanged in this session	6
17	conversation_history	JSON	Full conversation history stored as JSON array	[{"sender": "user", "text": "Halo"}, ...]
18	created_at	TIMESTAMP	Session creation timestamp	2026-02-20 9:15:00
19	updated_at	TIMESTAMP	Last update timestamp	2026-02-20 9:25:00
20	last_activity	TIMESTAMP	Timestamp of the last user activity	2026-02-20 9:25:00
21	expires_at	TIMESTAMP	When the session will automatically expire (300s timeout)	2026-02-20 9:30:00
22	closed_at	TIMESTAMP	Timestamp when the session was closed	NULL

x. User_profile_data

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each profile	901
2	phone_number	VARCHAR	WhatsApp phone number (one profile per user)	6281234567890
3	skill_level	VARCHAR	User's technical skill level: 'newbie', 'intermediate', 'advanced', 'expert'	intermediate
4	device_type	VARCHAR	Type of device used by the user	Samsung Galaxy A14
5	device_specs	JSON	Detailed device specifications in JSON format	{"os": "Android 13", "ram": "4GB"}
6	time_availability	VARCHAR	User's time availability: 'urgent', 'short', 'flexible'	flexible
7	preferred_language	VARCHAR	Preferred language for communication	id
8	communication_style	JSON	Communication preferences (explanation depth, technical terms, detail level)	{"detail_level": "simple"}
9	total_interactions	INTEGER	Total number of interactions/sessions	12
10	completed_issues	INTEGER	Number of successfully resolved issues	9
11	failed_issues	INTEGER	Number of issues that failed to resolve	2

12	success_rate	DOUBLE PRECISION	Ratio of completed to total issues	0.75
13	avg_satisfaction	DOUBLE PRECISION	Average satisfaction rating across all interactions	4.2
14	is_frustrated	BOOLEAN	Whether the user is currently flagged as frustrated	FALSE
15	frustration_level	DOUBLE PRECISION	Frustration level on a 0–1 scale	0.15
16	frustration_keywords_count	INTEGER	Number of frustration-indicating keywords detected	1
17	full_profile	JSON	Complete serialized user profile in JSON format	{"skill": "intermediate", "interactions": 12, ...}
18	created_at	TIMESTAMP	Profile creation timestamp	2026-01-15 8:30:00
19	updated_at	TIMESTAMP	Profile last update timestamp	2026-02-20 9:30:00
20	last_interaction	TIMESTAMP	Timestamp of the user's most recent interaction	2026-02-20 9:30:00

xi. Solution_attempts

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each solution attempt	401
2	conversation_id	INTEGER	Reference to the parent conversation	101
3	phone_number	VARCHAR	WhatsApp phone number	6281234567890
4	solution_id	VARCHAR	Unique identifier for the solution from the knowledge base	gps_restart_01
5	category	VARCHAR	Issue category (e.g., GPS, Camera, Battery)	GPS
6	problem_description	TEXT	Description of the problem being solved	GPS tidak bisa connect sejak pagi
7	solution_steps	JSON	JSON array of step-by-step solution instructions	["Buka Settings", "Tap GPS", "Turn off lalu on"]
8	kb_match_score	DOUBLE PRECISION	Similarity score between the problem and the KB solution (0–1)	0.92
9	outcome	VARCHAR	Result of the attempt: 'worked', 'partially_worked', 'failed',	worked

			'abandoned', 'escalated'	
10	user_feedback	TEXT	User's textual feedback about the solution	Ya, GPS sudah normal sekarang
11	time_to_implement_minutes	DOUBLE PRECISION	Time taken by the user to implement the solution in minutes	5.5
12	follow_up_attempts	INTEGER	Number of follow-up questions asked by the user	1
13	escalation_needed	BOOLEAN	Whether the issue required escalation to human support	FALSE
14	ai_confidence	DOUBLE PRECISION	AI's confidence in the provided solution (0–1)	0.87
15	user_skill_level	VARCHAR	User's skill level at the time the solution was offered	intermediate
16	user_frustration	DOUBLE PRECISION	User's frustration level at the time (0–1)	0.20
17	meta_data	JSON	Additional metadata in JSON format	{"kb_version": "2.1"}
18	created_at	TIMESTAMP	Solution attempt creation timestamp	2026-02-20 9:20:00
19	updated_at	TIMESTAMP	Timestamp when the outcome was updated	2026-02-20 9:27:00
20	outcome_recorded_at	TIMESTAMP	Timestamp when the outcome was specifically recorded	2026-02-20 9:27:00

xii. Solution_effectiveness

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each effectiveness record	1101
2	solution_id	VARCHAR	Unique identifier of the solution	gps_restart_01
3	category	VARCHAR	Issue category for this solution	GPS
4	total_attempts	INTEGER	Total number of times this solution was attempted	48
5	worked_count	INTEGER	Number of attempts where	35

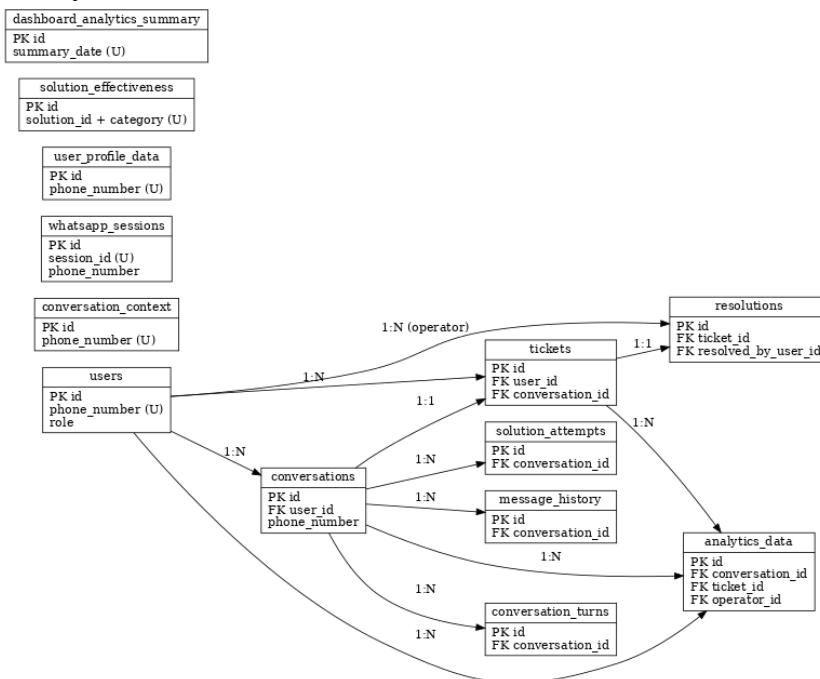
			the solution fully worked	
6	partially_worked_count	INTEGER	Number of attempts where the solution partially worked	5
7	failed_count	INTEGER	Number of attempts where the solution failed	4
8	abandoned_count	INTEGER	Number of attempts that were abandoned by the user	2
9	escalated_count	INTEGER	Number of attempts that required escalation	2
10	success_rate	DOUBLE PRECISION	Combined success rate: (worked + partially_worked) / total	0.83
11	pure_success_rate	DOUBLE PRECISION	Pure success rate: worked / total	0.73
12	escalation_rate	DOUBLE PRECISION	Rate of escalation: escalated / total	0.04
13	avg_implementation_time	DOUBLE PRECISION	Average time to implement the solution in minutes	6.2
14	avg_ai_confidence	DOUBLE PRECISION	Average AI confidence when providing this solution	0.85
15	avg_user_satisfaction	DOUBLE PRECISION	Average user satisfaction rating (1–5)	4.1
16	health_score	DOUBLE PRECISION	Composite health score (0–1) based on all metrics	0.82
17	recommendation	VARCHAR	Solution recommendation status: 'excellent', 'good', 'okay', 'needs_review', 'broken'	good
18	last_30_days_attempts	INTEGER	Number of attempts in the last 30 days	12
19	last_30_days_success_rate	DOUBLE PRECISION	Success rate in the last 30 days	0.90
20	trending	VARCHAR	Trend direction: 'up', 'stable', 'down'	up
21	meta_data	JSON	Additional metadata in JSON format	{"last_review": "2026-02-15"}

22	created_at	TIMESTAMP	Record creation timestamp	2026-01-20 0:00:00
23	updated_at	TIMESTAMP	Record last update timestamp	2026-02-20 9:28:00

xiii. Dashboard_analytics_summary

No	Field Name	Data Type	Description	Example
1	id	INTEGER	Unique identifier for each summary record	1201
2	summary_date	DATE	The date this summary represents (one record per day)	2026-02-20
3	total_conversations	INTEGER	Total number of conversations on this date	23
4	total_tickets_created	INTEGER	Total number of tickets created on this date	8
5	avg_resolution_time	INTEGER	Average ticket resolution time in minutes	18
6	ai_success_rate	DOUBLE PRECISION	AI chatbot success rate for the day (0–1)	0.72
7	operator_count	INTEGER	Number of active support operators on this date	3
8	most_common_category	VARCHAR	The most frequently reported issue category	GPS
9	avg_user_satisfaction	DOUBLE PRECISION	Average user satisfaction rating for the day	4.0
10	created_at	TIMESTAMP	Record creation timestamp	2026-02-20 23:59:00

c. Entity tables



d. Relations with cardinality of each Relation

i. User – Conversation Relationship

- Cardinality: One-to-Many (1:N)
- Explanation: One user can have multiple conversations with the system.
- Example: A driver reports different issues on different days, resulting in multiple conversations.

ii. User – Ticket Relationship

- Cardinality: One-to-Many (1:N)
- Explanation: One user can create multiple support tickets, but each ticket belongs to only one user.
- Example: A driver submits tickets for GPS issues and camera issues separately.

iii. User – Resolution Relationship

- Cardinality: One-to-Many (1:N)
- Explanation: One support operator can resolve many tickets, while each resolution is handled by one operator.
- Example: A support operator resolves multiple tickets in a day.

iv. Conversation – Message Relationship

- Cardinality: One-to-Many (1:N)
- Explanation: One conversation consists of multiple messages exchanged between the user and the system.
- Example: A conversation includes user messages and chatbot responses.

v. Conversation – Ticket Relationship

- Cardinality: One-to-One (1:1)
 - Explanation: One conversation may result in one support ticket when escalation is required.
 - Example: A conversation that cannot be resolved by AI generates a single ticket.
- vi. Conversation – Conversation Turn Relationship**
- Cardinality: One-to-Many (1:N)
 - Explanation: One conversation is composed of multiple conversation turns representing interaction steps.
 - Example: Greeting, problem description, solution attempt, and confirmation steps.
- vii. Conversation – Solution Attempt Relationship**
- Cardinality: One-to-Many (1:N)
 - Explanation: One conversation can involve multiple solution attempts suggested by the system.
 - Example: The system suggests restarting GPS and updating firmware in the same conversation.
- viii. Conversation – Analytics_Data Relationship**
- Cardinality: One-to-Many (1:N)
 - Explanation: One conversation can generate multiple analytics records used for reporting and trend analysis.
 - Example: A conversation produces analytics such as resolution time, AI success rate, and issue category count.
- ix. Ticket – Resolution Relationship**
- Cardinality: One-to-One (1:1)
 - Explanation: Each ticket has exactly one resolution record.
 - Example: A ticket is resolved either by AI or by a support operator.
- x. Ticket – Analytics Data Relationship**
- Cardinality: One-to-Many (1:N)
 - Explanation: One ticket can generate multiple analytics records for reporting and evaluation.
 - Example: Resolution time and category statistics are recorded for one ticket.

STANDALONE ENTITIES (no Foreign Key Relationships)

- xi. Conversation Context**
- Cardinality: One-to-One (1:1)
 - Explanation: Each phone number has one conversation context record to maintain the current state and issue details.
 - Example: A driver's phone number stores the latest issue state and description during an active conversation.
- xii. WhatsApp Sessions**
- Cardinality: One-to-Many (1:N) from user perspective

- Explanation: One user can have multiple WhatsApp sessions representing different reporting interactions.
- Example: A driver starts multiple reporting sessions for different vehicle issues.

xiii. User Profile Data

- Cardinality: One-to-One (1:1)
- Explanation: Each user has one profile record containing personalization and interaction history data.
- Example: A driver has a profile with skill level, device information, and satisfaction metrics.

xiv. Solution Effectiveness

- Cardinality: One-to-Many (1:N) from solution perspective
- Explanation: One solution effectiveness record aggregates results from many solution attempts.
- Example: A GPS restart solution aggregates success and failure results from multiple attempts.

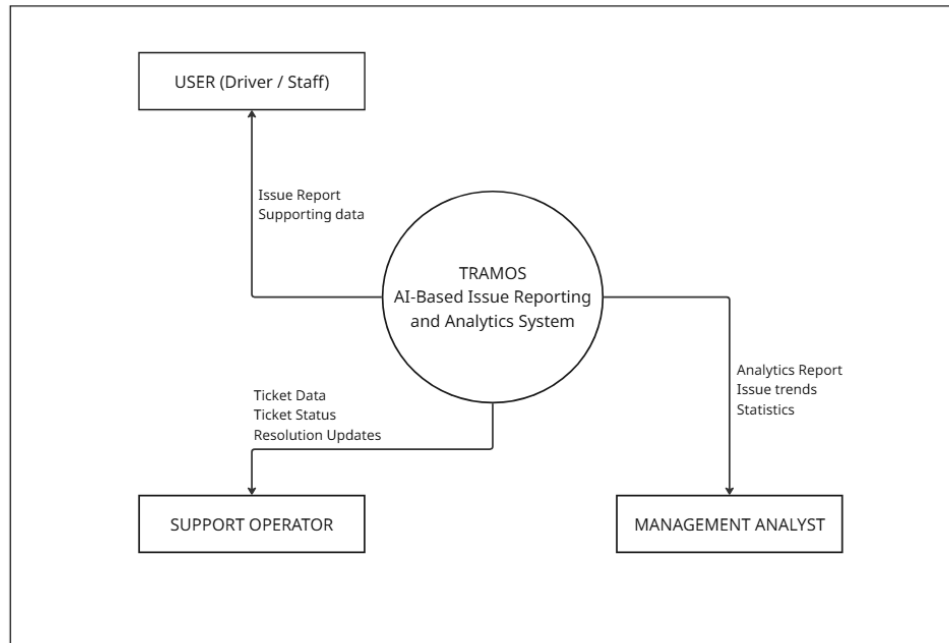
xv. Dashboard Analytics Summary

- Cardinality: One-to-One (1:1) per date
- Explanation: Each record represents a daily summary of system-wide analytics metrics.
- Example: A daily report summarizes total conversations, tickets, and AI success rate.

4. Functional Requirement

a. Context Diagram

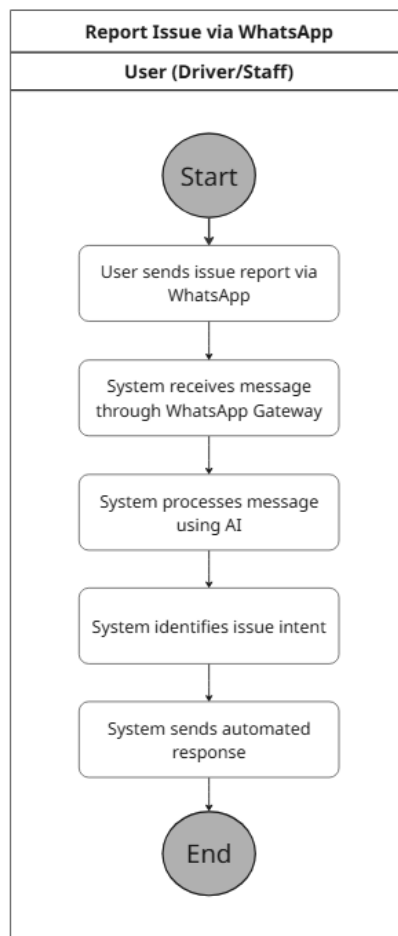
This diagram shows the system boundaries and the relationships between the system and external entities that interact directly. In the TRAMOS AI-Based Issue Reporting and Analytics System, there are three main external entities, namely Users(Drivers/Staff), Support Operators, and Management Analyst.



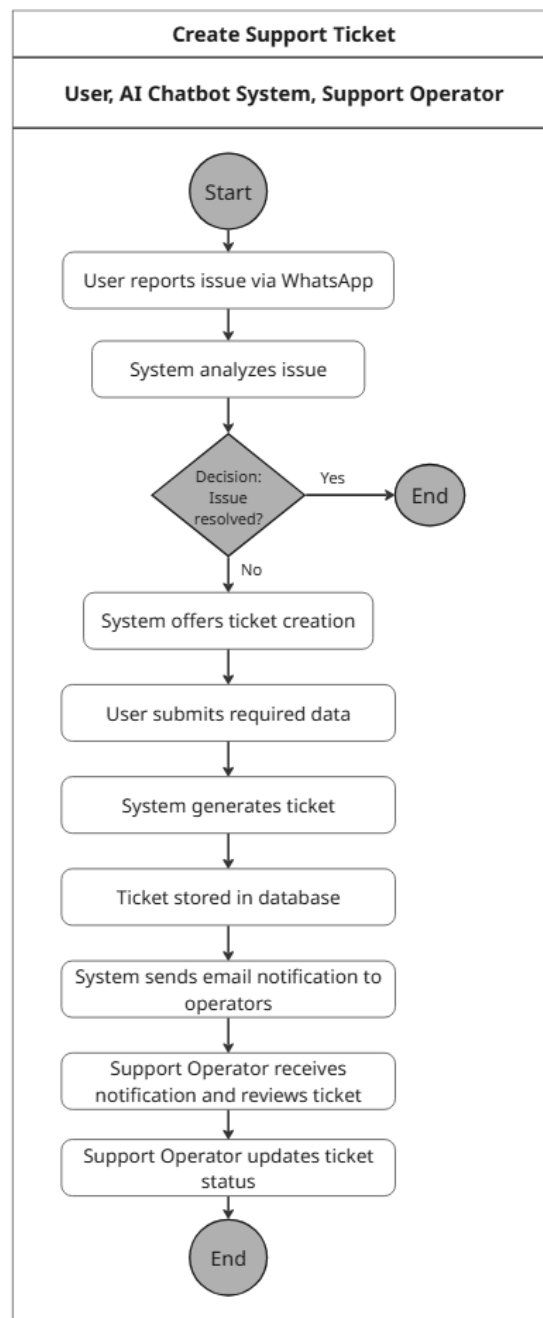
b. Activity Diagram and Data Flow Diagram

i. Activity Diagram

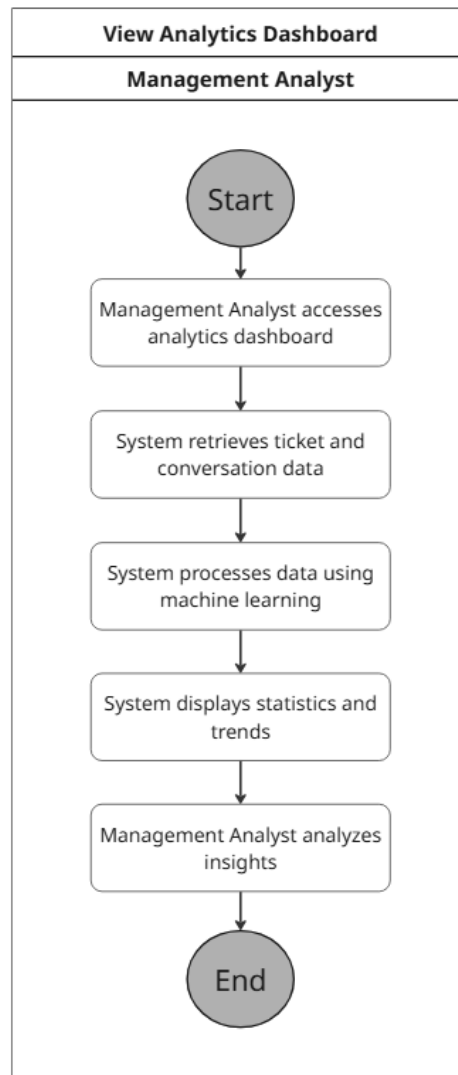
• Report Issue via WhatsApp



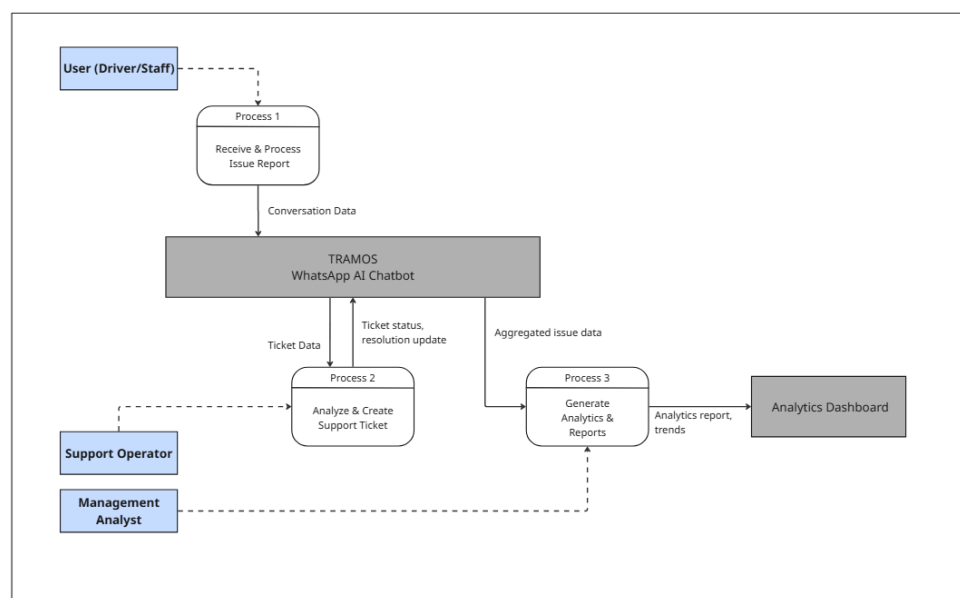
- **Create Support Ticket**



- **View Analytics Dashboard**



ii. Data Flow Diagram



c. Describe in detail the process inside the Activity Diagram and Data flow diagram

The functional processes of the proposed system are described based on the activity diagrams and data flow diagram that illustrate the operational flow of issue reporting, ticket management, and analytics generation in the TRAMOS AI-Based Issue Reporting and Analytics System.

The first process begins with the User (Driver/Staff) reports an issue via WhatsApp. This process starts with the user sending a message that describes the issue faced during the operational activities. The system receives the message from the WhatsApp Gateway and processes it with the AI engine. At this point, the system initiates intent analysis to determine the type of issue that is being reported. After the intent analysis, the system automatically sends a response to the user. This process is mainly focused on managing common issues through self-service. All the conversation data created during this conversation is stored in the conversation database for proper documentation and analysis purposes in the future.

If the reported problem cannot be handled from the automated response, the system moves to the second process, which is the support ticket creation process. In this process, the system analyzes the problem that has not been solved and gives the user an option to create a support ticket. Once the user agrees, the system asks for the necessary supporting information, which may include details about the problem and other relevant operational details. After the user provides the necessary information, the system automatically generates a support ticket and stores it in the ticket database. System also automatically sends email notification to Support Operator, the support ticket is then made accessible to the Support Operator, who examines the support ticket using the ticketing system dashboard. The Support Operator then updates the status of the support ticket and the progress of the resolution until the problem is solved.

The third process is related to analytics and reporting, which helps in decision-making for managers. In this process, the system combines information from both the conversation database and the ticket database. The combined information is then processed using machine learning algorithms to analyze patterns, trends, and common problems. Based on the analysis, the system provides statistical reports and insights, which are then shown through the analytics dashboard. The Management Analyst uses the dashboard to analyze issue trends, performance, and operational insights. The insights are then used to analyze system performance, common problems, and actions to take to correct or prevent them.

Overall, the activity diagrams and data flow diagram show how the system takes raw issue reports and converts them into organized tickets and valuable analytical insights. The combination of automated issue resolution, organized ticket

management, and analytical insights ensures that the system is running efficiently and with minimal reliance on manual processing.

5. Non-Functional Requirement

The non-functional requirements define the quality characteristics and operational constraints of the proposed system. These requirements ensure that the proposed system operates efficiently, securely, and reliably in a real-world environment.

a. **Performance**

The system must be able to process user messages and generate automated responses within an acceptable response time. The response time of the chatbot must not take more than a few seconds under normal network conditions to ensure a smooth user experience.

b. **Availability**

The system must be available to users on a 24/7 basis, considering that operational issues may occur at any time. The system downtime must be minimized to ensure uninterrupted support to users in the field.

c. **Reliability**

The system must be able to store conversation data, ticket data, and analytics data without losing any data. All transactions, such as ticket generation and status updates, must be consistently recorded in the database.

d. **Security**

The system must be able to maintain the confidentiality and integrity of user data. The ticketing system and analytics dashboard must be restricted based on user roles, and sensitive data must be secured using authentication and authorization processes.

e. **Usability**

The system should be user-friendly and require less training for users. The WhatsApp-based interface is designed to conform to user behavior, enabling users to submit issue reports without needing to learn a new application.

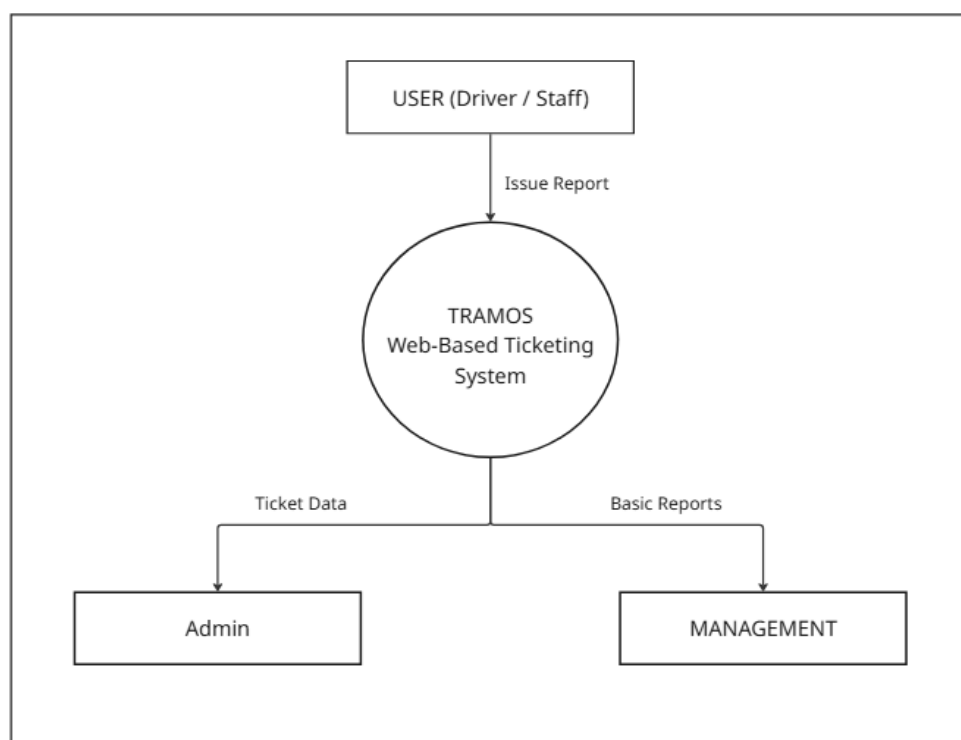
f. **Scalability**

The system should be scalable to support an increasing number of users and issue reports without compromising performance. The system architecture should support future scaling, including the addition of new analytical tools.

D. ALTERNATIVE SOLUTION

Alternative Solution 1: Web-Based Manual Ticketing System with Basic Reporting

In this alternative solution, the TRAMOS Web System was developed by adding a structured web-based problem reporting module. This solution focuses on improving documentation and reducing dependence on manual communication through private chat. Based on the block diagram provided, the *TRAMOS Web-Based Ticketing System* serves as the central component of the entire ticketing system by connecting users, support operators, and management through a single workflow.



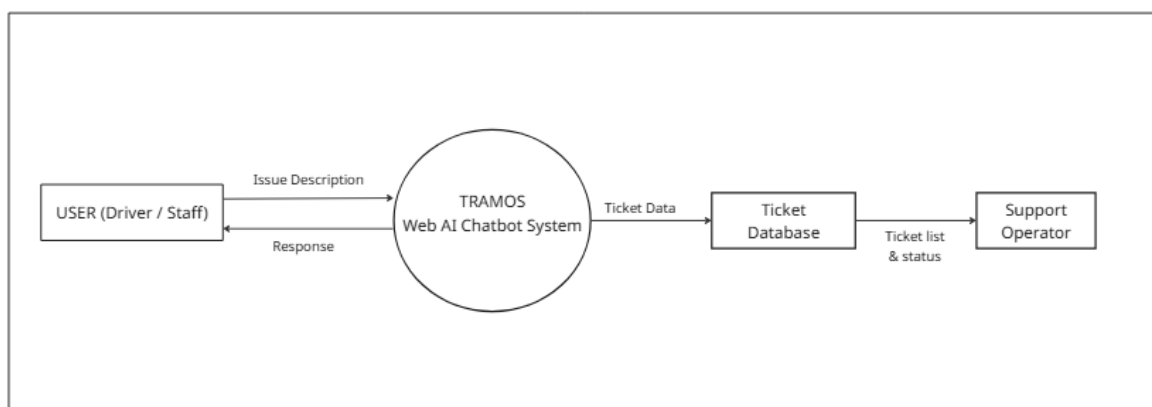
The process begins when users, such as drivers, or staff, access the TRAMOS website and fill out the problem reporting form provided. The form contains basic information such as the title of the problem, description of the complaint, the problem category, and other supporting data. Each submitted report is automatically recorded in the system database as a support ticket, then the admin can view the tickets submitted through the ticketing dashboard, which allows them to manage the entire process of resolving the issues according to the needs of the system's opera

The system also provides basic reporting feature that can be used to generate simple reports such as a report describing the total number of tickets submitted within a certain period, a report describing the distribution of tickets according to their categories, etc. This reporting feature allows the management to have a general overview of the issues submitted to the

system. Overall, this solution is more focused on improving documentation while minimizing the use of manual chat communication through a relatively simple system architecture.

Alternative Solution 2: Web-Based AI ChatBot with Ticketing System

The *Web-Based AI Chatbot with Ticketing System* proposes the integration of an intelligent chatbot directly within the TRAMOS Web environment to support the automation of issue reporting and initial problem handling. This approach has been proposed with the goal of minimizing human intervention by allowing users to interact with the AI-powered chatbot via the web environment to report and manage issues. Based on the block diagram provided, it is evident the AI Chatbot will be the primary component of the proposed system.



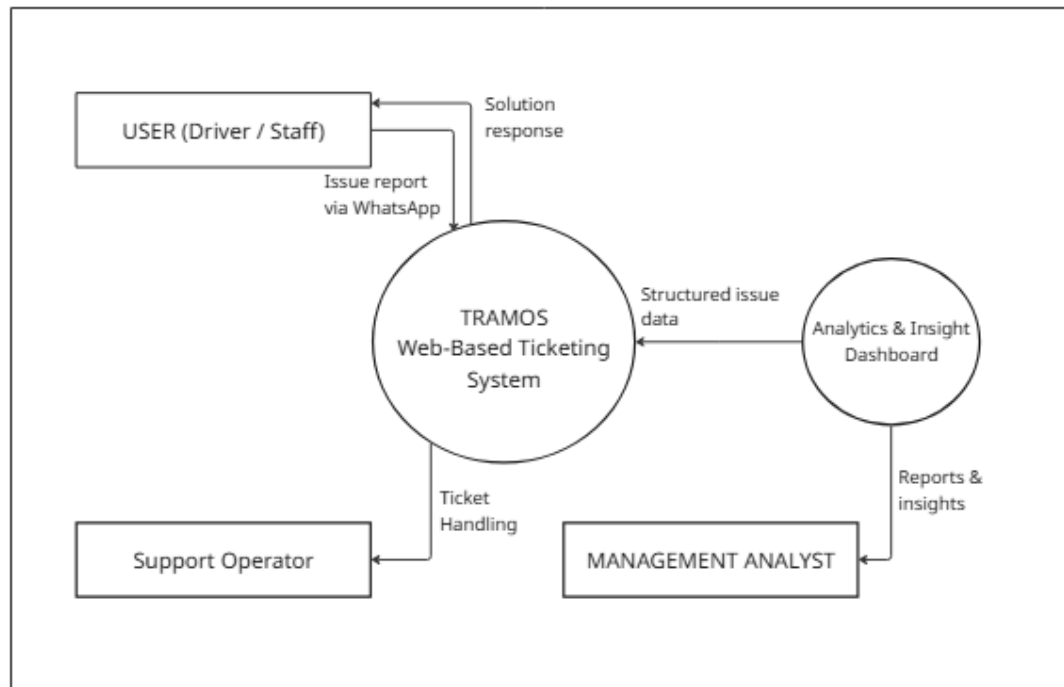
In this alternative solution, the users are required to access the TRAMOS web platform and submit the issues through the AI chatbot interface instead of using manual chat communication. The chatbot uses the intent detection approach to analyze the input provided by the user and resolve the issue by retrieving the relevant information from the pre-defined knowledge base. If the issue can be resolved automatically, the system is able to provide an immediate response to the user.

If the chatbot is unable to resolve the issue automatically through the use of the knowledge base, the system is able to generate a support ticket for the user's issue by collecting the relevant information from the user through the chatbot's conversational interface. The ticket is then stored in the system database and made available to support operators via the ticketing dashboard. Support operators review, update, and resolve the tickets manually as required. Although the solution is highly automated compared to the original solution, it is still limited and may not be very helpful for highly mobile users in the field.

Alternative Solution 3 : AI WhatsApp Chatbot Integrated with Analytics and Insights Dashboard

The AI WhatsApp Chatbot Integrated with Analytics and Insights Dashboard is designed to offer an end-to-end automated issue reporting, ticketing, and analytics solution for TRAMOS Web. This alternative uses WhatsApp as the main of communication for users such as driver

to report issues using a platform they are most likely to be comfortable using. Based on the block diagram provided, this alternative combines the AI WhatsApp chatbot with a ticketing system to offer the required functionality for monitoring operations using the analytics dashboard



In this alternative solution, users are able to report problems directly to the system via WhatsApp, without the need to manually access the TRAMOS Web interface. Incoming messages are handled by a WhatsApp Gateway and then analyzed by the AI chatbot, which analyzes user intent and issues. When possible, the chatbot offers immediate solutions based on the TRAMOS troubleshooting knowledge base. If not possible, the chatbot starts an automated ticket creation process by gathering necessary information from the user and creating a formatted support ticket.

All conversation logs and ticket information are stored in a centralized database and constantly analyzed by the analytics module. The analytics and insights dashboard translates raw ticket information into useful visual information, such as issue frequency, resolution time, problem patterns, and operational trends. Support operators are given access to the ticketing system for status and resolution updates, while management analysts are given access to the dashboard for performance analysis and opportunities for preventive and corrective measures. This alternative solution has the highest level of accessibility, scalability, and analytics capability among all the proposed solutions and is thus ideal for real-world logistics and fleet management operations.

REFERENCES

- Rao, S., Patel, V., & Kumar, A. (2023). Scalability issues in centralized manual communication systems for service industries. *Computers & Industrial Engineering*, 175, 108854.
- Gupta, A., & Sharma, R. (2022). Predictive analytics for issue prevention in logistics platforms. *IEEE Transactions on Intelligent Transportation Systems*, 23(8), 11234–11245.
- Laudon, K. C., & Laudon, J. P. (2023). *Management information systems: Managing the digital firm* (17th ed.). Pearson Education.
- Al-Hawari, F., & Barham, H. (2019). A machine learning–based help desk system for IT service management. *International Journal of Advanced Computer Science and Applications*, 10(4), 1–12.
- Wijaya, A. S. (2024). Machine learning approaches for helpdesk ticketing systems: A review. *Journal of Theoretical and Applied Information Technology*, 102(5).
- Jetir. (2025). Online chatbot-based ticketing system: AI-powered helpdesk solution for automated ticket generation. *Journal of Emerging Technologies and Innovative Research*.
- Rachman, F., Setiawan, F. A., & Harimas, A. (2025). Chatbot implementation for digital reporting systems using WhatsApp. In *Proceedings of the Open Society Conference (OSC)*.
- Truss, M., & Boehm, S. (2024). AI-based classification of customer support tickets: State of the art and implementation with AutoML. *arXiv*.
- Mauliana, P., Wiguna, W., & Permana, A. Y. (2020). Pengembangan e-helpdesk support system berbasis web. *Jurnal Responsif*, 2(1), 19–29.
- Siahaan, M., & Kelvin. (2023). Pengembangan helpdesk ticketing system berbasis website dengan menggunakan metode SDLC, XP, dan Scrum. *KLIK: Kajian Ilmiah Informatika dan Komputer*, 4(3), 1399–1410.
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson Education.
- Pressman, R. S., & Maxim, B. R. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.
- Meta Platforms. (2024). *WhatsApp Business Platform API documentation*.
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*.
- osTicket Project. (2024). *osTicket documentation*.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv*.
- IEEE. (2024). IEEE software requirements specification standards. *IEEE Xplore*.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... Thomas, D. (2001). Manifesto for agile software development. *The Agile Alliance*.
- Papazoglou, M. P., & van den Heuvel, W. J. (2007). Service oriented architectures: Approaches, technologies and research issues. *The VLDB Journal*, 16(3), 389–415.
- Erl, T. (2016). *Service-oriented architecture: Concepts, technology, and design*. Pearson Education.
- Papazoglou, M. P., & van den Heuvel, W. J. (2007). Service oriented architectures: Approaches, technologies and research issues. *The VLDB Journal*, 16(3), 389–415.

Erl, T. (2016). *Service-oriented architecture: Concepts, technology, and design*. Pearson Education.